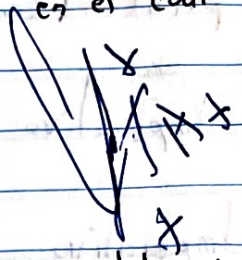
Unidad 2

28-09-26

Modelo de programación funcional

La programación funcional no es más que un paradigma de programación, es decir, es una forma en el cual podemos resolver diferentes problemáticas.



Funcional Programming
(Programación Funcional),
Comúnmente abreviado FP.

Cuando nos encontramos desarrollamos software utilizando este paradigma, trabajando principalmente con funciones, evitando los datos mutables, así como el hecho de compartir estados entre funciones.

La programación funcional es un paradigma declarativo. Nos enfocaremos en "qué" estamos haciendo y no en "cómo" se está haciendo que sería el enfoque imperativo. Esto quiere decir que nosotros expresemos nuestra lógica sin describir controles de flujo; no usaremos ciclos o condicionales.

Ejemplo utilizando Java

Java, como sabemos es un lenguaje orientado a objetos, sin embargo, en versiones recientes nosotros podemos hacer uso de la programación funcional.

En este ejemplo, la problemática es conocer la cantidad de elementos en la lista mayores a 10.

```
List<Integer> numeros = List.of(18,6,4,15,55,78,12,9,8);
```

Nosotros podemos resolver el problema utilizando ciclos y condicionales. Este sería el enfoque imperativo.

```
// Declarativo.
List<Integer> numeros = List.of(18,6,4,15,55,78,12,9,8);
int contador = 0;
for(int numero : numero){
    if(numero>10){
```

```
        contador ++;
    }
}
System.out.println(contador);
```

El enfoque imperativo sería el siguiente

```
// Imperativo
Long result = numero.stream().filter(num -> num > 10).count();
System.out.println(result);
```

Como observamos, con el enfoque imperativo las líneas de código se reducen y nuestro código se torna más legible.
En esta última implementación podemos concluir que es funcional, ya que delega el control de flujos y condiciones a funciones (filter, sum).

Conceptos:

- Funciones puras
- Composición de funciones
- Estados compartidos
- Mutabilidad
- Efecto secundario

Funciones Puras: Las funciones puras, no son más que funciones, las cuales, dado el mismo input, siempre retornan el mismo output, además de no tener efectos secundarios

UPAK

Composición de funciones

La composición de funciones es el proceso de combinar dos o más funciones, teniendo como finalidad ejecutar cada una de estas funciones en secuencia para obtener un resultado en concreto.

> f(x)

> g(x)

> f(g(x))

En este caso combinamos la función f de x con la función g de x.

Estado compartido

El estado compartido es cualquier variable, objeto o espacio de memoria que exista en un ámbito compartido. Un ámbito compartido puede incluir el alcance global o ámbitos de cierre. A menudo, en la programación orientada a objetos, los objetos se comparten entre ámbitos al agregar propiedades a otros objetos.

```
Materia materia = new Materia ();
Usuario usuario = new Usuario ();

usuario.materia = materia;
```

Mutabilidad

Un objeto inmutable es aquel que no puede ser modificado una vez haya sido creado. Por ejemplo, una variable en Java, no es un objeto inmutable.

```
String caracteres = "ABC";
String nuevaCadena = caracteres.toLowerCase();
```

Podemos ver a los objetos inmutables como constantes, una vez nosotros asignamos un valor este no podrá ser modificado.

Efectos Secundarios

Un efecto secundario es cualquier cambio de estado en la aplicación que sea observable fuera de la función llamada.

UPAM

```
public static int funcion (int x ) {
Variable_global = 10; // Cambio de valor
return x + 10;
}
```

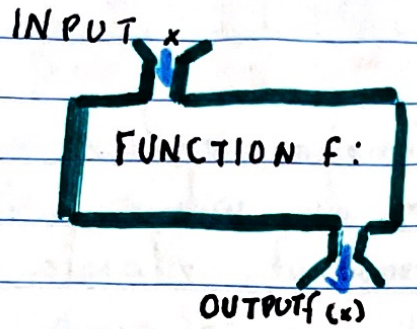
En este caso mi "función", no es una función pura, ya que esta posee efectos secundarios, modifica el valor de una variable externa a la función.
Los efectos secundarios se evitan principalmente en la programación funcional, para tener como resultado programas mucho más fáciles de comprender y fáciles de probar.

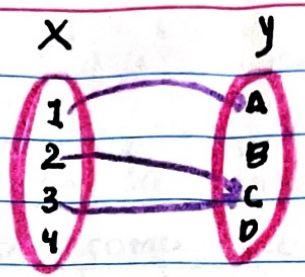
Lenguajes de programación

- Java
- PHP
- Ruby
- Python
- Elixir
- Kotlin
- Haskell
- Erlang

Introducido en 1930 por Alonzo Church, pero fue hasta 1958, con la especificación de Lisp que surgió el primer lenguaje con una porción funcional.
Como su nombre lo indica, está centrada en funciones, las cuales son tratadas como ciudadanos de primera clase.
En este esquema, la función siempre representará la relación entre una entrada y una salida, tal como en matemáticas, en las que tienes que si a una función introduces un valor x, obtendrás como resultado un valor Y.

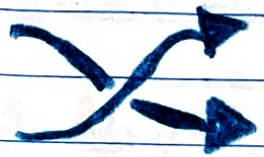
UPAK





Otro concepto conocido como los fonciones de orden superior, que son aquellas funciones que pueden tomar otras funciones como argumentos o retornar otras funciones como resultado de su ejecución.

3 Funciones que son populares en la programación funcional



Map Project



Reduce Fold



Filter Select

En el caso de map que recibe una función que especifica la transformación de un valor a otro.
El caso de reduce que nos sirve para aplicar una función de acumulación a una colección de elementos, y filter que toma una función que especifica los criterios de elegibilidad de los elementos de una colección.

28. 09 - 26

Cálculo lambda de Alonzo Church

Es un sistema matemático creado en la década de 1930 para estudiar de manera formal como se pueden representar y ejecutar funciones. Es una de las bases teóricas de la computación y de la programación funcional.
En el cálculo lambda prácticamente todo se expresa mediante funciones. Sus tres elementos principales son:

1.- **Variables:** representan valores, por ejemplo: x
2.- **Abstracción:** Sirven para definir una función.
por ejemplo: $\lambda x. x$
Significa una función que recibe x y devuelve el mismo valor. Es equivalente a: $f(x) = x$
3.- **Aplicación:** consiste en ejecutar una función con un argumento. Por ejemplo: $(\lambda x. x)\ 5$
El resultado es: 5
Otro ejemplo es: $(\lambda x. x + 1)\ 4$
Se sustituye x por 4: $4 + 1 = 5$
Esta sustitución se conoce como reducción beta.

El cálculo lambda fue desarrollado por Alonzo Church y tuvo una gran importancia porque permitió establecer formalmente qué significa que un problema sea computable. Además, está estrechamente relacionado con la máquina de Turing: ambos modelos pueden representar el mismo tipo de cálculos.
Su influencia puede observarse actualmente en lenguajes de programación funcional como Haskell, Lisp y Scheme, y también en funciones anónimas o expresiones lambda utilizadas en lenguaje como Python.
El cálculo lambda es un modelo matemático basado en funciones, variables y sustitución que permite representar procesos de cálculo y constituye una de las primeras

bases teóricas de la programación funcional

Elemento	¿Qué es?	Forma general	Ejemplo	Explicación
Variable	Representa un dato o valor.	x, y, z	x	La letra x representa un valor.
Abstracción	Define una función.	$\lambda x. M$	$\lambda x. x$	Es una función que recibe x y devuelve x.
Aplicación	Ejecuta una función usando un argumento.	M N	$(\lambda x. x)$ 5	Se sustituye x por 5, dando como resultado 5.

Ejemplo completo:

$(\lambda x. x + 1)$ 4

- Variables: x
- Abstracción: $\lambda x. x + 1$
- Aplicación: $(\lambda x. x + 1)$ 4
- Resultado: 5

Función Recursiva

06-10-26

Erlang

Es un lenguaje de programación de propósito general creado originalmente en el laboratorio de ciencias de la computación de Ericsson. Fue diseñado principalmente para desarrollar sistemas que necesitan funcionar continuamente, manejar muchas operaciones al mismo tiempo y recuperarse de errores sin detener todo el sistema.
Erlang es considerado principalmente un lenguaje de programación funcional y destaca especialmente por su capacidad para trabajar con concurrencia, sistemas distribuidos y tolerancia a fallos.

Características

Concurrencia: permite ejecutar una gran cantidad de procesos al mismo tiempo.
Tolerancia a fallos: si un proceso falla, el resto del sistema puede continuar funcionando.
Sistemas distribuidos: diferentes partes de una aplicación pueden ejecutarse en diferentes servidores y comunicarse entre ellas.
Paso de mensaje: los procesos se comunican enviándose información entre ellos.

Erlang se utiliza especialmente en telecomunicaciones, servidores, mensajería instantánea, comercio electrónico, sistemas bancarios y aplicaciones en tiempo real.
Ericsson, por ejemplo, lo han utilizado en infraestructura de telecomunicaciones.

Ejemplo

```
-module(hola).
-export([saludar/0]).

saludar() ->
    io:format("Hola mundo~n").
```

Este programa crea un módulo llamado hola y una función llamado saludar, que imprime Hola mundo

Elixir

Es un lenguaje de programación funcional creado por José Valim. Su desarrollo comenzó como un proyecto de investigación y desarrollo dentro de Plataformatec, y la versión 1.0 fue publicada en septiembre de 2014.

Una características muy importante es que Elixir se ejecuta sobre la máquina virtual de Erlang, conocida como BEAM. Por eso puede aprovechar muchas de las características que hicieron famoso Erlang, como la concurrencia, escalabilidad y tolerancia a fallos.

Elixir fue diseñado buscando que el desarrollo de aplicaciones fuera más productivo y que el código fuera fácil de mantener, sin perder las capacidades de Erlang.

Características

- Es un lenguaje funcional
- Utiliza la máquina virtual BEAM
- Permite crear muchos procesos ligeros
- Ofrece concurrencia
- Tiene de mecanismos de tolerancia a fallos
- Facilita la construcción de sistemas distribuidos

Ejemplo

```
defmodule Hola do
  def Saludar do
    IO.puts ("Hola mundo")
  end
end
```

Después de ejecutase:

```
Hola.Saludar ()
```

Y mostrará

Hola mundo

Diferencia principal

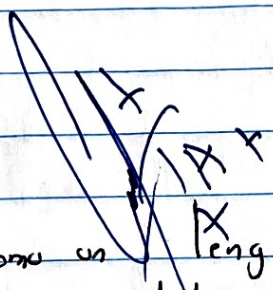
Aunque los dos lenguajes utilican el ecosistema de Erlang y la maquina virtual BEAM, su sintaxis es diferente.

En Erlang:

```
suma (A, B) ->
    A + B.
```

Elixir

```
def suma (a, b) do
    a + b
end
```



Elixir puede verse como un lenguaje más moderno que aprovecha mucho de las capacidades desarrolladas originalmente para Erlang

¿Dónde utilica Erlang y Elixir en WhatsApp?

WhatsApp ha utilizado principalmente Erlang en el backend de sus servidores, ya que permite manejar una gran cantidad de usuarios y mensajes al mismo tiempo

Erlang se utilica para:

* Mantener muchas conexiones simultáneas.
* Procesar y entregar mensajes.
* Manejar notificaciones
* Evitar que un fallo afecte todo el sistema
* Permitir que la plataforma pueda crecer y atender millones de usuarios

06-10-26

Apuntes

PLF

Java
HTML
Nodejs
Haskell

Python
JavaScript
Ruby
Prolog

Backend

Java
Python
JavaScript
Nodejs

Ruby
Rust

Frontend

HTML
JavaScript

Hilos
Procesos

Investigar

Funciones de orden funcional

Map ←
filter ←
reduce
forEach
Sort

Fold/reduce ←

Joana

13-10-26

Investigar Funciones de Orden Funcional (Map, filter y fold / Reduce)

Las funciones de orden superior son aquellas que pueden recibir otras funciones como argumentos o devolver una función como resultado. Son fundamentales dentro del paradigma de programación funcional porque permite procesar datos de manera más sencilla, reutilizable y organizada. En lenguaje como Haskell, Python y Java Script, estas funciones permiten trabajar con listas y colecciones sin necesidad de escribir ciclos repetitivos.

Entre las funciones de orden superior más utilizadas se encuentra Map, Filter y Fold / Reduce, las cuales permiten transformar, seleccionar y acumular elementos de una lista.

Función Map

Definición

La función map se utiliza para aplicar una función a cada elemento de una lista, generando una nueva lista con los resultados obtenidos.

Su principal característica es que transforma los elementos sin modificar la cantidad de elementos de la lista original.

Características

- Aplica una misma operación a todos los elementos
- Devuelve una nueva lista.
- Conserva el orden y la cantidad de elementos.
- Evita escribir ciclos manualmente

Ejemplo en Haskell

```
duplicar :: [Int] -> [Int]
duplicar lista = map (*2) lista
```

Prueba en GHCi:

ghci> duplicar [1,2,3,4,5]

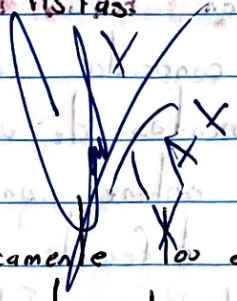
[2,4,6,8,10]

Explicación

La función map recorre (recorre) la lista y multiplica cada número por 2. Por ejemplo, el número 1 se convierte en 2, el 2 en 4 y así sucesivamente.

Aplicaciones

Convertir temperaturas, aumentar precios, transformar textos y realizar operaciones matemáticas sobre listas.



Función Filter

Definición

La función filter permite seleccionar únicamente los elementos de una lista que cumple una condición determinada. Los elementos que no cumplen dicha condición son descartados.

A diferencia de map, esta función puede generar una lista con menos elementos que el original.



Características:

- Evalúa una condición para cada elemento.
- Conserva únicamente los elementos que cumplen la condición
- No modifica los valores seleccionados.
- Es útil para buscar y clasificar información

Ejemplo en Haskell

```
numerosPares :: [Int] -> [Int]
numerosPares lista = filter even lista
```

Prueba en GHCi:

ghci> numerosPares [1,2,3,4,5,6,7,8]

[2,4,6,8]

Explicación
La función filter evalúa cada número mediante even, que determina si es par. Si el número cumple la condición, permanece en la lista; de lo contrario, se elimina del resultado.
Aplicaciones
Seleccionar estudiantes aprobados, filtrar productos por precio, identificar números positivos y buscar registros que cumplan determinados criterios.

Función Fold / Reduce

La función fold, conocida como reduce en otros lenguajes, permite combinar los elementos de una lista mediante una operación acumulativa para obtener un resultado.
En Haskell existen diferentes variantes, como foldl (plegado desde la izquierda) y foldr (plegado desde la derecha).

Características

- Procesa una lista acumulando resultados.
- Utiliza una función de combinación y un valor inicial.
- Puede obtener sumas, productos y otros resultados.
- Permite reducir una colección a un único valor o estructura.

Ejemplo en Haskell

```
SumarLista :: [Int] -> Int
SumarLista lista = foldl (+) 0 lista
```

Prueba en GHCi:

```
ghci > sumarlista [1, 2, 3, 4, 5]
15
```

Explicación

La función foldl comienza con un acumulador de valor 0 y suma cada elemento de la lista.

Operación	Acumulador
0 + 1	1
1 + 2	3
3 + 3	6
6 + 4	10
10 + 5	15

El resultado final es 15.

Aplicación

Calcular totales de ventas, obtener promedios, contar elementos, multiplicar valores y acumular información.

Diferencias entre Map, Filter y Fold / Reduce

Función	Propósito	Resultado
map	Transformar cada elemento	Lista transformada
filter	Seleccionar elementos que cumple una condición	Lista filtrada
fold / reduce	Combinar elementos mediante una acumulación.	Valor o estructura acumulada

Lunes 19 Octubre

Bucles

Java 8 en adelante

$$\sum_{i=1}^{n=5} i = 15$$

Ejemplo 5

$1+2+3+4+5 = 15$

Función que crea funciones se utiliza (Recursividad)

Funciones [ ] Listo

Sum [1 ..... n]

$\sum 1+2$

$\sum 1+3$

Imperativo

- Count = 0 "contador"
- n total = 0 "Variable"

función para sumar → (repetir)
count := count + 1
total := total + count
until count = n = 5

global python: Alcance de la variable

count = 0
total = 0
n = 5

Donde se almacena (en la suma)

```
def sumar (count, total, n):
    for count, in range (n):
        count <- = count + 1
        total <- = total + count
        return total
print (sumar (count, total, n)
```

UPAK

Características	Funciones prefijas	Funciones infijas	Funciones postfijas
Definición	El nombre de la función aparece antes de sus argumentos.	El operador o función aparece entre dos argumentos	El operador aparece después de su argumento.
Sintaxis	función x y	x operador y	(x operador)
Posición	Al principio	En medio	Al final.
Ejemplo en Haskell	Suma 5 3	5 + 3	(5!)
Resultado	8	8	120 (factorial)
Uso principal	Aplicar funciones matemáticas y operaciones	Realizar operaciones aritméticas y comparaciones	Representar Operaciones como el factorial
Ventaja	Es sencilla y permite aplicar funciones directamente.	Facilita la lectura de operaciones matemáticas.	Permite escribir algunas operaciones de forma más natural.
Desventaja	Puede resultar menos familiar al principio	Requiere conocer la precedencia de operadores	Necesita la extensión PostfixOperators para operadores unarios postfijos.
Ejemplos comunes	sqrt 25, max 84	8+4, 10*2	(5!), (4!)
Compatibilidad	Haskell permite usarla directamente, escribiendo primero el nombre de la función y después sus argumentos	Haskell la admite directamente para operaciones matemáticas y comparaciones	Haskell necesita la extensión PostfixOperators de GHC para usar operadores unarios después de su arg.
Cantidad de argumentos	Puede recibir uno o varios argumentos, dependiendo de la función	Generalmente utilizados argumentos, uno de cada lado del operador.	Utiliza un argumento colocado antes del operador
Aplicaciones en programación	Se utiliza para cálculos, manejo de listas y procesamiento de datos.	Se emplea en operaciones aritméticas, comparaciones	Puede utilizarse en operaciones especiales, como factorial

Unidad 2 04/11/26

Modelo de Programación Funcional

Intervalos, Operadores y Evaluación perezosa en Haskell

Haskell es un lenguaje de programación funcional que permite desarrollar mediante funciones y expresiones matemáticas. Una de sus principales características es que utiliza la evaluación perezosa, lo que significa que las operaciones se realizan únicamente cuando sus resultados son necesarios.

2.3 Intervalos en Haskell

Los intervalos en Haskell son una forma de generar secuencias de valores automáticamente, sin necesidad de escribir cada elemento por separado. Se utilica principalmente para crear listas de números o caracteres que sigue un orden determinado.

Tipos de intervalos

1.- **Intervalo simple**: genera una lista de valores desde un número inicial hasta uno final. Ejemplo: numeros = [1..10]

Resultado: [1,2,3,4,5,6,7,8,9,10]

2.- **Intervalo con incremento**: permite establecer cuánto aumentará cada número de la lista. Ejemplo: pares = [2,4,..20]

Resultado: [2,4,6,8,10,12,14,16,18,20]

3.- **Intervalo descendente**: genera valores de mayor a menor utilizando un incremento negativo. Ejemplo: descendente = [10,9..1]

Resultado: [10,9,8,7,6,5,4,3,2,1]

4.- **Intervalo de caracteres**: permite generar una secuencia de letras.

Ejemplo: letra = ['a'..'f']

Resultado: "abcdef"

5.- **Intervalo infinito**: generar una secuencia que no tiene un límite definido.

Ejemplo

naturales = [1..]

Se puede utilizar take para obtener unicamente los primeros valores:

take 5 naturales

Resultado: [1,2,3,4,5]

04-11-26

2.4 Operadores en Haskell

Los operadores son símbolos que permiten realizar operaciones matemáticas, comparación y evaluaciones lógicas entre diferentes valores.

Operadores aritmeticos

Se utilizan para realizar cálculos matemáticos.

Operador	Función	Ejemplo	Resultado
+	Suma	5+3	8
-	Resta	10-4	6
*	Multiplicación	4*3	12
/	División fraccionaria	10/2	5.0
div	División entera	10 div 3	3
mod	Residuo de división	10 mod 3	1
^	Potencia	2^3	8

Operadores de comparación

Permite comparar dos valores y devuelven un resultado verdadero (True) o falso (False).

Operador	Significado	Ejemplo	Resultado
==	Igual	5==5	True
/=	Diferente	5/=5	True
>	Mayor que	8>3	True
<	Menor que	2<6	True
>=	Mayor o igual	5>=5	True
<=	Menor o igual	3<=7	True

Operadores lógicos

Se utilizan para comprobar una o varias condiciones.

- && significa Y: ambas condiciones deben ser verdaderas.

04 - 11 - 26

- || significa O: al menos una condición debe ser verdadera.
- not significa NO: cambia verdadero por falso o viceversa.

Ejemplo en haskell

5 > 3 && 10 > 5

Resultado: True

Porque las dos condiciones son verdaderas.

Sirven para hacer cálculos, comparar información y tomar decisiones dentro de un programa.

2.7.- Evaluación perezosa en Haskell

La evaluación perezosa significa que Haskell solamente realiza un cálculo cuando necesita conocer su resultado.

Es decir, no realiza todas las operaciones desde el principio, sino que calcula lo que se necesita en ese momento.

Ejemplo sencillo

Imaginemos que tenemos una lista de números de 1 a 1000, pero solamente queremos los primeros cinco números.

```
take 5 [1..1000]
```

Resultado:

```
[1, 2, 3, 4, 5]
```

Haskell solamente necesitó generar los primeros cinco números, porque son los únicos que pedimos.

No fue necesario generar los 1000 números completos.

Ventaja

- Evita realizar cálculos innecesarios
- Permite trabajar con listas infinitas

Desventaja

- En algunos casos puede consumir más memoria
- Puede ser difícil saber cuando se realiza un cálculo.